VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Natural Language Processing

CO3086

Assignment Technical Report

Source Code

LEGAL CONTRACT INFORMATION EXTRACTION & SEMANTIC ANALYSIS

Instructor:	Mr. Nam Duong Ho	
Students:	Cuong Doan Phuong Hung	ID 2310381
	Khiem Ton Vo Hoai	ID 2211576
	Phong Ha Thuc Thanh	ID 2212549

HO CHI MINH CITY, 26 APRIL 2026

Contents

I Processing and Syntax Analysis **2**

1 Dataset Overview **2**

2 Clause Splitting **3**
2.1 Objective and Problem Understanding 3
2.2 Preprocessing of Legal Text 4
2.3 Clause Extraction Algorithm 5
2.4 Qualitative Evaluation 7

3 Noun Phrase Chunking **9**
3.1 The Linguistic Challenge in Legal Text 9
3.2 Methodology: The IOB Tagging Schema 10
3.3 Qualitative Evaluation 12

4 Dependency Analysis **13**
4.1 Objective and Syntactic Mapping 13
4.2 Methodology: The Universal Dependencies Framework 13
4.3 Data Representation and JSON Schema 15
4.4 Technical Evaluation 16

Abstract

Part I

Processing and Syntax Analysis

1 Dataset Overview

Dataset Selection & Justification

For this project, the primary data source is the **Atticus Open Contract Dataset (CUAD)**. CUAD is a specialized, expert-annotated corpus containing 510 commercial legal contracts. It was created specifically to benchmark Natural Language Processing (NLP) models in the legal domain and provides a highly authoritative collection of unstructured contract text.

While the corpus encompasses over 500 contracts, a full-scale analysis was not necessary for demonstrating the effectiveness of the proposed pipeline. Many of the documents – such as boilerplate Non-Disclosure Agreements – are highly repetitive in structure and lack the complex linguistic content best suited for rigorous evaluation of clause segmentation and extraction algorithms.

Alignment with Assignment Objectives

The selected evaluation documents provide intentionally diverse linguistic challenges that directly correspond to the core milestones of this project. Rather than processing the entire dataset, three contracts were curated to stress-test distinct aspects of the NLP pipeline:

- **Sponsorship Agreement**
  (*iPayment, Inc. and Wells Fargo Bank, N.A.*)
  Syntactic Complexity for Clause Splitting (Section 2): This contract contains multi-tiered definitions and elaborate termination provisions. It is designed to rigorously evaluate the dependency-aware clause segmenter, particularly its handling of subordinating conjunctions and intricate legal enumerations without fragmenting semantic units.

- **Distributor Agreement**
  (*Wireless Links Inc. and Jaguar Investments, Inc.*)
  Technical Density for Noun Phrase Chunking (Task 1.2): This document is rich in technical multi-word noun phrases such as "G3001 GPSLink mobile terminal" and "MidLink Middleware software." These serve as an exacting test for the IOB tagging procedure required for accurate noun phrase chunking.

- **Software License and Maintenance Agreement**
  (*SFG Financial Corp. and 551 FX IB Associates, LLC*)

Feature-Rich Entities for Custom NER (Task 2.1): This contract includes a high frequency of intricate numerical and temporal entities—such as precise monetary amounts ($63,309), variable rates ("15% rebate," "tier-based pricing"), and deadlines. These features ensure the NER model is evaluated on a wide array of entity formats.

Additionally, all three selected contracts provide:

- **Explicit Intents for Classification (Task 2.3):** Clear examples of contractual Obligations ("shall pay"), Prohibitions ("shall not disclose"), and Rights ("reserves the right to"), yielding an unambiguous ground truth for TF-IDF and transformer-based intent classifiers.
- **Structural Depth for RAG Implementation (Assignment 3):** Extensive appendices, schedules, and cross-referenced sections offer a realistic test bed for Retrieval-Augmented Generation, requiring high-precision retrieval of specific fee schedules and penalty stipulations.

This deliberately engineered subset, spanning a spectrum of commercial contract types and linguistic features, ensures comprehensive evaluation of clause segmentation, NER, classification, and retrieval capabilities.

2 Clause Splitting

2.1 Objective and Problem Understanding

Legal contracts typically consist of long, complex sentences that include multiple clauses connected through conjunctions, conditions, and enumerations. These structures make downstream Natural Language Processing (NLP) tasks – such as entity recognition or semantic role labeling – significantly more difficult. Therefore, the goal of this task is to transform raw contract text into a sequence of semantically independent clauses, where each clause represents a meaningful and analyzable unit.

To achieve this, the implementation focuses on designing a hybrid rule-based and dependency-aware clause segmentation algorithm that leverages both surface-level punctuation and syntactic structure.

Approach Overview:

- **Rule-Based Splitting:** Initial clause candidates are extracted using heuristics based on surface features such as semicolons, periods, and conjunctions. These are strong signals of clause and sentence boundaries in legal English.

- **Dependency Parsing:** For challenging cases where punctuation is absent or ambiguous, a syntactic dependency parser identifies clausal structures in complex sentences.
- **Hybrid Postprocessing:** The outputs from both strategies are further refined. For example, enumerations and subordinate clauses attached to the main clause are separated to ensure semantic independence.

This combination of shallow and deep linguistic cues helps ensure that each segment produced is both syntactically and semantically meaningful for downstream tasks.

2.2 Preprocessing of Legal Text

Before clause splitting, the raw contract text is subjected to a dedicated preprocessing stage designed to clean and normalize the data. Legal documents, particularly those sourced from public filings, frequently contain artifacts such as redaction markers, extraneous page headers/footers, and inconsistent whitespace. If left unaddressed, these features can negatively impact the performance of subsequent NLP analyses – especially clause segmentation.

```python
def clean_legal_text(text):
  """Removes redactions, page numbers, and formatting artifacts."""
  text = re.sub(r'\[\*\*\*\]', '', text)
  text = re.sub(r'Page \d+', '', text)
  text = re.sub(r'\n+', ' ', text)
  text = re.sub(r'\s+', ' ', text)
  return text.strip()
```

The preprocessing function applies several key transformations:

- **Removal of redaction markers:** Textual placeholders such as `[***]`, which indicate redacted or confidential content, are stripped from the document to prevent confusion with actual clause content.
- **Elimination of page numbering:** Regular expressions are used to identify and remove page headers or footers (e.g., "Page 1") that appear throughout scanned or OCR-processed contracts.
- **Whitespace normalization:** Multiple consecutive newline (`\\n`) and space characters are replaced with a single space, ensuring that sentence and clause boundaries are not obscured by formatting noise.

Through these steps, the input text is transformed into a linguistically consistent form that is more amenable to parsing, tokenization, and clause boundary detection. The preprocessing routine helps maximize the precision and recall of the clause splitter by minimizing spurious tokens and misplaced boundaries in the downstream analysis.

2.3 Clause Extraction Algorithm

At the core of the system is the `extract_clauses()` function, responsible for identifying clause boundaries within the preprocessed legal text. The method utilizes spaCy's NLP pipeline to perform tokenization and dependency parsing, iterating sequentially over each token to dynamically determine appropriate split points based on linguistic and syntactic cues. To handle the unique complexities of legal texts, the algorithm is divided into three specialized heuristic rules.

Algorithm 1 Dependency-Aware Clause Splitting

```
procedure ExtractClauses(raw_text)
    doc ← PreprocessAndParse(raw_text)
    clauses ← [ ], start_pos ← 0
    for each token in doc do
        split ← false
        if token ∈ {".",";"} then                                  ▷ Rule 1: Hard Punctuation
            split ← true
        else if token = ":" and Next(token) = "(" then             ▷ Rule 2: Look-Ahead
            split ← true
        else if token.dep = "cc" and IntroducesNewSubject(token) then   ▷ Rule 3: Syntactic
            split ← true
        end if
        if split then
            span ← Text from start_pos to token (inclusive)
            PostProcessAndStore(clauses, span)
            start_pos ← position after token
        end if
    end for
    return clauses
end procedure
```

Token Whitespace Preservation

A critical challenge in NLP text segmentation is the **Tokenization Trap**. When parsers like spaCy process text, they split punctuation marks, symbols, and words into discrete tokens. Naively reconstructing a clause by joining these tokens with spaces ruins the original formatting, yielding artifacts such as `( i )` instead of `(i)` or "V / MC Systems" instead of "V/MC Systems".

In the legal domain, preserving exact formatting is paramount. To resolve this, the algorithm abandons token-level string joining. Instead, it maintains index pointers (`start_idx` and `end_idx`) and slices the original document span (`doc[start_idx:end_idx].text`). Because spaCy spans retain the exact character offsets of the raw input, this slicing technique guarantees 100% fidelity to the original contract formatting, eliminating artificial whitespace while ensuring

clean data for downstream tasks.

Handling Legal Enumerations

Standard sentence boundary detection models are trained on general English, where periods indicate the end of a thought. However, legal contracts frequently utilize complex enumerations, where massive lists of conditions or obligations are chained together using semicolons (;). A standard tokenizer will fail on these structures, grouping multiple distinct conditions into a single, unmanageable paragraph.

To address this, the algorithm explicitly defines the semicolon as a hard clause boundary. This isolates each condition into its own independent string.

Standard Parser Output (Failure):

> *"Such Reserve Account may be funded by: (i) one or more debits; (ii) one or more deductions; or (iii) ISO's delivery of a letter of credit."*
> (Treated as one massive clause).

Our Algorithm Output (Success):

Clause 1. Such Reserve Account may be funded by: (i) one or more debits;

Clause 2. (ii) one or more deductions;

Clause 3. or (iii) ISO's delivery of a letter of credit.

By splitting explicitly at semicolons, the algorithm preserves the semantic independence of each list item. This is critical for Task 2.3 (Intent Classification), as it allows the classifier to evaluate each condition individually rather than attempting to classify a convoluted block of text.

Syntactic Dependency Rules

For complex sentences where punctuation is insufficient or ambiguous (such as clauses connected by "and", "or", or "but"), the algorithm leverages dependency parsing to determine if a split is syntactically valid. Simply splitting at every coordinating conjunction (`cc`) results in severe over-splitting, destroying compound adjectives or compound predicates (e.g., splitting "Approval will not be withheld or delayed" into two meaningless fragments).

To prevent this, a strict **"New Subject" Heuristic** was implemented. The algorithm will only split a sentence at a conjunction if *all* of the following syntactic conditions are met:

- The token is a coordinating conjunction (`dep_ == "cc"`).

- Its head is a verb (`pos_ == "VERB"`).
- The head verb is connected to a secondary verb (`dep_ == "conj"`).
- **Critical Check:** The secondary verb possesses its own explicit nominal subject (`nsubj` or `nsubjpass`).

Example of the heuristic in action:

> "In the event a Merchant refuses to consent, ISO will notify SERVICERS, and Bank will have the right to terminate its sponsorship."

The parser detects the conjunction "and", identifies "will notify" and "will have" as the two connected verbs, and verifies that "Bank" is the explicit subject of the second verb. Because a new subject is present, it is a truly independent clause, and the algorithm safely cleanly severs the string before the conjunction.

2.4 Qualitative Evaluation

The effectiveness of the Dependency-Aware Clause Splitter is evaluated by comparing the raw OCR-processed text of the Sponsorship Agreement against the final segmented output. This qualitative assessment focuses on three primary linguistic successes: governing clause isolation, enumeration precision, and syntactic integrity.

```
10.3  Termination by ISO. ISO may terminate this Agreement prior to its expiration for cause upon prior written notice
to SERVICERS as follows:

  (a)   Upon a Material breach of any representation, warranty or covenant in this Agreement by SERVICERS which is not
  cured by SERVICERS within sixty (60) days of receipt of written notice from ISO; provided however, that if SERVICES
  are attempting in good faith to cure such breach within said sixty (60) day cure period but the nature of the breach
  prevents a cure within sixty (60) days, then SERVICERS shall be allotted an additional number of days to cure,
  provided the total number of days of the cure period shall be no greater than one hundred twenty (120) days;

  (b)   Upon any failure by SERVICERS to pay any amount due under this Agreement which does not give rise to the right
  to terminate under any other provision of this Agreement within five (5) Business Days after written notice to
  SERVICERS of its failure to pay the amount;

  (c)   Upon the insolvency or bankruptcy of either of the SERVICERS;

  (d)   Upon the termination of the Service Agreement for any reason;

  (e)   Upon a Material adverse change in the business or financial condition of either of SERVICERS;

  (f)   Upon an assignment of this Agreement by SERVICERS without ISO's prior written consent;

  (g)   Upon the failure by SERVICERS to maintain good standing with Visa or a MasterCard;

  (h)   If required by a governmental or regulatory body or agency or by any Bank Card association to terminate;

  (i)   Upon cessation of Bank Card operations by SERVICERS; or
```

Figure 2.1: Original OCR-based Sponsorship Agreement

Figure 2.1 presents a raw, unsegmented snapshot of Section 10.3 from the Sponsorship Agreement as it appears post-OCR processing. This image serves as a *high-difficulty baseline*, reflecting the dense, non-linear structure typical of commercial legal filings.

The section is anchored by a "governing clause" that transitions directly into a complex enumeration of ten distinct termination triggers, each separated by semicolons and frequently containing nested conditions – notably the "provided however" block in item (a). This input exemplifies the so-called "Tokenization Trap": conventional sentence splitters typically fail, either merging all ten items into a single run-on clause or fragmenting the logic of subordinate phrases (such as conditions within enumerated items).

By adopting this particular passage as the starting point, the evaluation below underscores the limitations of off-the-shelf sentence segmentation and illustrates the necessity of the proposed hybrid, dependency-aware heuristic to preserve both syntactic boundaries and conditional logic.

```
377   10.3 Termination by ISO.
378   ISO may terminate this Agreement prior to its expiration for cause upon prior written notice to SERVICERS as follows:
379   (a) Upon a Material breach of any representation, warranty or covenant in this Agreement by SERVICERS which is not cured by SERVICERS within
      sixty (60) days of receipt of written notice from ISO;
380   provided however, that if SERVICES are attempting in good faith to cure such breach within said sixty (60) day cure period but the nature of
      the breach prevents a cure within sixty (60) days, then SERVICERS shall be allotted an additional number of days to cure, provided the total
      number of days of the cure period shall be no greater than one hundred twenty (120) days;
381   (b) Upon any failure by SERVICERS to pay any amount due under this Agreement which does not give rise to the right to terminate under any
      other provision of this Agreement within five (5) Business Days after written notice to SERVICERS of its failure to pay the amount;
382   (c) Upon the insolvency or bankruptcy of either of the SERVICERS;
383   (d) Upon the termination of the Service Agreement for any reason;
384   (e) Upon a Material adverse change in the business or financial condition of either of SERVICERS;
385   (f) Upon an assignment of this Agreement by SERVICERS without ISO's prior written consent;
386   (g) Upon the failure by SERVICERS to maintain good standing with Visa or a MasterCard;
387   (h) If required by a governmental or regulatory body or agency or by any Bank Card association to terminate;
388   (i) Upon cessation of Bank Card operations by SERVICERS;
389   or (j) At any time upon one hundred eighty (180) days prior written notice to the SERVICERS with no penalty.
```

Figure 2.2: Clause Splitting Output

2.5.1 Success in Structural Segmentation

As evidenced by the IDE line numbers in Figure 2.2, each clause is successfully isolated onto an independent line, proving that the segmentation logic correctly terminates strings at identified boundaries.

- **Governing Clause Isolation:** The algorithm correctly applied Rule 2 (Look-Ahead) at Section 10.3. It identified the colon after "as follows:" and severed the string before the start of the enumeration (a).
- **Semantic Independence:** By isolating the governing intent ("ISO may terminate...") from the specific triggers, the pipeline ensures the Intent Classifier (Task 2.3) can distinguish between a "Right" and its "Conditions".

2.5.2 Handling Deep Enumerations

The evaluation confirms a high degree of precision in handling Rule 1 (Hard Punctuation) boundaries within legal lists.

- **Triggers (a) through (j):** Each termination condition is extracted as a distinct semantic unit.
- **Data Balancing:** Splitting at the semicolon prevents the creation of a "heavy first item," which occurs when introductory text is erroneously merged with the first list entry. This leads to more balanced feature vectors for downstream machine learning tasks.

2.5.3 Syntactic Robustness

The most rigorous test of the New Subject Heuristic (Rule 3) appears in clause (a), which contains a complex "provided however" condition.

- **Constraint Success:** Despite the presence of multiple potential split points – including "and", "but", and "then" – the algorithm correctly maintained the entire 60-day to 120-day cure period as a single, contiguous clause.
- **Logic:** Because the algorithm requires a new nominal subject to trigger a split, it recognized these fragments as dependent modifiers of the primary breach rather than independent actions.

Through these results, the system demonstrates that it can navigate the dense formatting of the Atticus Open Contract Dataset while preserving 100% fidelity to the original text via span slicing.

3 Noun Phrase Chunking

Following the isolation of semantically independent clauses in Task 1.1 (Section 2), the next critical phase in the NLP extraction pipeline is **Noun Phrase (NP) Chunking**. The primary objective of this task is to identify and extract non-recursive noun phrases from the segmented text, representing their boundaries using the standard **Inside-Outside-Beginning (IOB)** tagging schema.

3.1 The Linguistic Challenge in Legal Text

In general domain text, entities are often single words or simple bigrams (e.g., "the company"). However, **commercial contracts exhibit extreme technical density**, making entity extraction significantly more complex. As highlighted in the dataset overview, documents such

as the Distributor Agreement frequently contain entities realized as complex, multi-word syntactic constituents – for example, "G3001 GPSLink mobile terminal" or "MidLink Middleware software."

If an NLP parser fails to recognize these multi-word strings as a single noun phrase, it will fracture the entity. For example, a naive parser might tag "mobile" as an adjective and "terminal" as a noun, losing the fact that the entire four-word string represents a single, indivisible hardware product governed by the contract.

Strategic Importance to the Pipeline

Noun Phrase Chunking is not merely a formatting exercise; it is a foundational prerequisite for the advanced machine learning objectives in Assignment 2:

- **Semantic Boundary Definition (Task 2.1: NER):** By grouping determiners, adjectives, and nouns into a single chunk, the algorithm establishes the exact token boundaries required to train a *Custom Named Entity Recognition* model on intricate entities (e.g., complex product names, precise numerical and temporal phrases).
- **Contextualizing Obligations (Task 2.3: Intent Classification):** While Clause Splitting (Task 1.1) isolates what the contractual rule is (the predicate), Noun Phrase Chunking isolates the *actors* and *objects* involved. It identifies who is obligated (e.g., "SERVICERS") and what specific object or payment is being acted upon (e.g., "Net Program Participation Fees").

By accurately mapping these phrases into the `Token \t Tag` tabular format required by the problem specification, the pipeline effectively transforms unstructured linguistic text into discrete, machine-readable objects, ready for entity classification and downstream retrieval.

3.2 Methodology: The IOB Tagging Schema

The objective of this module is to identify key entities – such as contracting parties, financial amounts, and legal deadlines – by labeling noun phrases (NPs) within extracted clauses. To structure the extracted clauses for downstream entity recognition, the pipeline implements the **Inside-Outside-Beginning (IOB)** tagging schema. This standard sequence-labeling format allows the system to represent the precise boundaries of multi-word noun phrases (NPs) at the token level.

Algorithm 2 IOB Noun Phrase Chunking with Boundary Reset

```
procedure GenerateIOBTags(clause)
    doc ← NLP_Parse(clause)
    tags ← array of "O" with length equal to number of tokens in doc
    for each chunk in doc.noun_chunks do
        start_new ← true
        for each token in chunk do
            if IsExcluded(token) then                    ▷ Filter punct, preps, etc.
                tags[token] ← "O"
                start_new ← true                         ▷ Reset NP boundary
            else
                tags[token] ← start_new ? "B-NP" : "I-NP"
                start_new ← false
            end if
        end for
    end for
    return FormatOutput(doc, tags)
end procedure
```

Dependency-Based Chunking Instead of using a simple Regular Expression (RegEx) parser, we implemented a methodology leveraging spaCy's dependency parser. This allows the system to:

- **Handle Complex Legal Syntax:** Legal text often contains nested modifiers. By utilizing the `noun_chunks` iterator, we identify phrases based on their syntactic "head" noun rather than just linear word order.
- **IOB Tagging Scheme Implementation:** We map the detected chunks to the required IOB format:
  - **B-NP (Beginning of Noun Phrase):** This tag is exclusively assigned to the very first token of a recognized noun phrase. It signals to the downstream model that a new entity or object boundary has started.
  - **I-NP (Inside a Noun Phrase):** This tag is assigned to all subsequent tokens that belong to the same noun phrase initiated by the B-NP tag.
  - **O (Outside):** This tag is assigned to any token that does not belong to a noun phrase. In legal contracts, this typically includes verbs, prepositions, coordinating conjunctions, and hard punctuation.

Refinement and Edge Case Handling To meet the specific requirements of legal document processing, the following logic adjustments were made:

- **Punctuation Filtering:** Standard legal punctuation (e.g., commas, periods, hyphens) is forced to the O tag to maintain clean boundaries between distinct legal entities.

- **Strict Boundary Resets:** We implemented a "reset" mechanism where any token following an O tag must start with B-NP if it belongs to a new noun phrase. This prevents "broken" sequences where an I-NP incorrectly follows an O tag.
- **Financial and Mathematical Symbols:** Special handling was applied to currency symbols (e.g., $) and operators (e.g., =) to ensure financial calculations and amounts remain semantically intact within their respective chunks.

3.3 Qualitative Evaluation

The effectiveness of the dependency-aware Noun Phrase Chunking module is evaluated by examining its ability to maintain syntactic continuity across complex legal ownership structures. Traditional shallow parsers frequently fracture multi-word entities when encountering punctuation, coordinating conjunctions, or possessive markers. To demonstrate the robustness of the implemented IOB tagging schema, the following sample from the *Sponsorship Agreement* illustrates the algorithm's handling of chained possessives and coordinated actors:

Token	Merchant	's	or	ISO	's	participation	in	MasterCard
IOB Tag	B-NP	I-NP	I-NP	I-NP	I-NP	I-NP	O	B-NP

Syntactic Continuity and Possessive Chaining As evidenced by the output above, the parser successfully maps the entire phrase `Merchant's or ISO's participation` as a single, contiguous semantic unit.

- **Possessive Integrity:** The algorithm correctly identifies that the possessive suffixes (`'s`) are not independent tokens to be excluded, but rather integral modifiers linking the contracting parties (`Merchant`, `ISO`) to the target object (`participation`). Tagging them as I-NP ensures the "Owner" and "Owned Object" remain syntactically bound.
- **Coordination Retention:** By tagging the conjunction `or` as I-NP, the pipeline avoids splitting the entity into two fragmented objects. This prevents `participation` from floating without its full ownership context, which is a vital prerequisite for accurately training the Named Entity Recognition model in Task 2.1.

Solving the "Broken Chunk" Problem Furthermore, the implementation guarantees strict sequence adherence through its boundary reset logic. In standard rule-based chunking, filtering out specific tokens (such as prepositions or hard punctuation) often leads to the "broken chunk" problem, where an I-NP tag incorrectly immediately follows an O tag, invalidating the entity boundaries for downstream machine learning models.

By dynamically resetting the beginning tag after any excluded token, the algorithm ensures that every newly initiated noun phrase strictly begins with a B-NP. The transition from

`participation` (I-NP) to `in` (O) and immediately to `MasterCard` (B-NP) proves that the extraction sequence is fully sanitized, resolving the broken chunk vulnerability and generating an artifact that is perfectly aligned with standard sequence-labeling requirements.

4 Dependency Analysis

4.1 Objective and Syntactic Mapping

The final component of the Assignment 1 NLP pipeline is **Dependency Analysis**. While Clause Splitting (Section 2) isolated the semantic boundaries of the text and Noun Phrase Chunking (Section 3) identified the specific entities involved, the objective of Dependency Analysis is to uncover the grammatical skeleton connecting them. This task extracts and maps the word-to-word *head-dependent* relationships for every token within a given clause, creating a structured tree of syntactic dependencies.

The Linguistic Utility in Legal English

Commercial contracts are notoriously dense, frequently employing passive voice, deep embedding, and vast distances between a governing verb and its associated subjects or objects. For instance, in a complex termination clause, a shallow keyword search or simple Part-of-Speech (POS) tagger might struggle to map the exact relationship between the acting party and the action due to the sheer volume of surrounding legal modifiers and conditions.

Dependency mapping resolves this ambiguity by translating the linear text into a directed graph of the sentence syntax. By identifying the root predicate (`ROOT` = "terminate") and tracing its directed edges to the nominal subject (`nsubj` = "ISO") and the direct object (`dobj` = "Agreement"), the pipeline exposes the core action of the clause regardless of word distance, intervening noise, or convoluted phrasing.

This token-level syntactic mapping is not only a formal requirement of the project but also a critical necessity for advanced relation extraction. It provides the exact structural logic required to populate the target JSON schema, enabling downstream machine learning models to programmatically understand who is doing what to whom within the contract.

4.2 Methodology: The Universal Dependencies Framework

To programmatically extract the grammatical relationships within the segmented clauses, the pipeline utilizes the **Universal Dependencies (UD)** framework. UD is a standardized, cross-linguistic annotation schema that categorizes syntactic relationships between a "head" token and its syntactic "dependents."

The Parsing Engine

The implementation leverages spaCy's `en_core_web_sm` model, which features a highly optimized, transition-based dependency parser. Unlike traditional constituency parsing – which breaks sentences into nested phrase brackets – transition-based dependency parsing establishes direct, directed arcs between individual words. During execution, the parser analyzes the topological structure of the clause and assigns a specific syntactic dependency label to every token. Key labels crucial for legal text analysis include:

- **ROOT**: The primary predicate or main verb of the clause (e.g., "terminate", "warrant", "pay"). All other words in the sentence ultimately trace their dependency back to this root.
- **nsubj / nsubjpass**: The nominal subject (or passive subject) performing the action, which usually identifies the obligated or acting party (e.g., "ISO", "SERVICERS").
- **dobj / pobj**: The direct object of a verb or the object of a preposition, typically identifying the target of an action or rule (e.g., "Agreement", "fees").
- **amod**: Adjectival modifiers that define the exact scope of legal entities (e.g., "prior" and "written" modifying "notice").

Handling Legal Modifiers and Complexity

Commercial contracts are characterized by extreme syntactic depth, heavily relying on chained prepositional phrases and clausal complements (e.g., "in accordance with the terms of this Agreement").

The dependency-based methodology excels in this environment because it systematically decomposes these chains without losing the governing logic. The parser links prepositions (`prep`) to their governing verbs or nouns, and then maps them to their subsequent prepositional objects (`pobj`). By mapping these relationships as a directed graph, the pipeline ensures that subordinate conditions and complex legal modifiers do not break the analytical chain. This allows the system to trace a complete syntactic pathway from a deep modifier all the way back to the governing `ROOT` verb, capturing the full structural reality of the clause.

Algorithm 3 Syntactic Dependency Extraction and JSON Serialization

```
procedure GenerateDependencyJSON(clauses)
    dataset_array ← [ ]
    for each clause in clauses do
        dependencies ← ExtractDependencies(clause)
        clause_record ← {"Clause": clause, "Dependencies": dependencies}
        Append(dataset_array, clause_record)
    end for
    return dataset_array
end procedure
procedure ExtractDependencies(clause)
    doc ← DependencyParse(clause)                    ▷ Generate directed syntactic graph
    token_mappings ← [ ]
    for each token in doc do
        if token.text.strip() ≠ "" then              ▷ Skip whitespace and empty tokens
            dep_entry ← {
                "Token": token.text,
                "Head": token.head.text,             ▷ String form of syntactic governor
                "Dependency": token.dep_label
            }
            Append(token_mappings, dep_entry)
        end if
    end for
    return token_mappings
end procedure
```

4.3 Data Representation and JSON Schema

To transform the directed acyclic graph (DAG) generated by the dependency parser into a serializable format, the system implements a mapping function that iterates through the parsed text. Each token is converted into a structured entry within a JSON array, ensuring that the syntactic hierarchy is preserved through explicitly defined `Head` pointers.

The Mapping Logic

- **Serialization:** The implementation captures the literal string of each token (`Token`) and its immediate syntactic governor (`Head`). By representing the head as its string form (not as a numeric index), the structure remains human-readable and easily verifiable.
- **Labeling:** The `Dependency` field utilizes standardized Universal Dependencies labels (such as `nsubj`, `ROOT`), explicitly codifying the grammatical relation for each token.
- **Noise Reduction:** Before converting to JSON, the extraction process skips over any non-semantic tokens (such as whitespace or stray newlines) using a simple filter (`if`

`token.text.strip()`:). This prevents OCR or formatting artifacts from polluting the output.

Output Structure

The final output aggregates this information into a nested JSON array, where each entry contains a segmented clause and a list of its token-level dependency mappings:

```
{
  "Clause": "ISO may terminate this Agreement",
  "Dependencies": [
    { "Token": "ISO",       "Head": "terminate",   "Dependency": "nsubj" },
    { "Token": "may",       "Head": "terminate",   "Dependency": "aux"   },
    { "Token": "terminate", "Head": "terminate",   "Dependency": "ROOT"  },
    { "Token": "this",      "Head": "Agreement",   "Dependency": "det"   },
    { "Token": "Agreement", "Head": "terminate",   "Dependency": "dobj"  }
  ]
}
```

This dictionary-based representation fulfills the data formatting requirements and provides a clean, machine-readable structure that supports both human audit and downstream automation for retrieval or relation extraction.

4.4 Technical Evaluation

The evaluation of the Dependency Analysis module focuses on two primary metrics: **syntactic fidelity** in complex legal sentences and **strict compliance** with the required JSON data schema.

```
{
  "Clause": "The confidential portion has been filed separately with the SEC.",
  "Dependencies": [
      { "Token":      "The",          "Head":      "portion",    "Dependency": "det"        },
      { "Token":      "confidential","Head":      "portion",    "Dependency": "amod"       },
      { "Token":      "portion",      "Head":      "filed",      "Dependency": "nsubjpass"  },
      { "Token":      "has",          "Head":      "filed",      "Dependency": "aux"        },
      { "Token":      "been",         "Head":      "filed",      "Dependency": "auxpass"    },
      { "Token":      "filed",        "Head":      "filed",      "Dependency": "ROOT"       },
      { "Token":      "separately",   "Head":      "filed",      "Dependency": "advmod"     },
      { "Token":      "with",         "Head":      "filed",      "Dependency": "prep"       },
      { "Token":      "the",          "Head":      "SEC",        "Dependency": "det"        },
      { "Token":      "SEC",          "Head":      "with",       "Dependency": "pobj"       },
      { "Token":      ".",            "Head":      "filed",      "Dependency": "punct"      }
  ]
}
```

Syntactic Fidelity and Structural Accuracy

Using the Sponsorship Agreement as a structural baseline, the parser correctly identifies the relational triad of actors, actions, and objects. In the parsed clause "ISO may terminate this

Agreement," the algorithm successfully designates `terminate` as the grammatical `ROOT`.

The dependency tree accurately maps the surrounding entities to this core predicate:

- **Nominal Subject (nsubj):** "ISO" is correctly identified as the acting party and assigned "terminate" as its syntactic head.
- **Direct Object (dobj):** "Agreement" is correctly identified as the target of the action, also pointing directly to the root verb.
- **Determiner (det):** The modifier "this" is correctly attached to its local head ("Agreement") rather than floating independently.

This proves that the transition-based parser reliably captures the core intent of a legal clause. By anchoring the acting party and the target object to the predicate, the system preserves the legal logic even when clauses are inverted or padded with complex modifiers.

Schema Compliance and Noise Robustness

The serialization module proved highly robust when converting the topological graph into the required flat JSON format. Every valid token is successfully mapped to the `Token`, `Head`, and `Dependency` keys without any data loss. Crucially, the implementation of the whitespace filtering heuristic effectively neutralized OCR layout artifacts. By enforcing a strict strip check before appending to the dictionary, stray newline characters, tab indents, and non-breaking spaces from the raw CUAD data were completely excluded from the final syntactic dataset.

The resulting JSON arrays provide a pristine, machine-readable foundation. This output strictly fulfills the final structural requirement of Task 1, cleanly concluding the Processing and Syntax Analysis phase and preparing the text pipeline for the Machine Learning and Classification objectives in Assignment 2.